01_data_collection

December 20, 2025

1 Credit Spread Prediction – Data Collection

This notebook downloads and cleans macro-financial data from FRED and Yahoo Finance.

[20]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import yfinance as yf
from fredapi import Fred
from curl_cffi import requests
```

[11]:
```
session = requests.Session(impersonate="chrome")
START_DATE = "1960-01-01"
END_DATE   = None  # or '2025-01-01'

FRED_API_KEY = "29d8cc35b9e6d25028c3de96e6782c6f"
fred = Fred(api_key=FRED_API_KEY)

# Helper to ensure tz-naive
def make_tz_naive(idx):
    if getattr(idx, "tz", None) is not None:
        return idx.tz_convert(None)
    return idx

def reindexing(df, start_date=START_DATE, end_date=END_DATE):
    idx = pd.date_range(start=start_date, end=end_date or pd.Timestamp.today(),␣
 ↪freq='D')
    idx = make_tz_naive(idx)
    df = df.reindex(idx)
    return df
```

[12]:
```
# Moody's Seasoned CCC Corporate Bond Yield
ccc = fred.get_series("BAMLH0A3HYCEY", observation_start=START_DATE,␣
 ↪observation_end=END_DATE, frequency="d")
ccc.index = pd.to_datetime(ccc.index)
ccc.name = "CCC_Yield"
ccc = ccc.reindex(pd.date_range(ccc.index.min(), ccc.index.max(), freq="D"))
ccc = ccc.interpolate(method='time')
```

```
# 10Y Treasury
t10 = fred.get_series("GS10", observation_start=START_DATE,␣
 ↪observation_end=END_DATE, frequency="m")
t10.index = pd.to_datetime(t10.index)
t10.name = "T10Y"
t10 = t10.reindex(pd.date_range(t10.index.min(), t10.index.max(), freq="D"))
t10 = t10.interpolate(method='time')
# t10 = t10.ffill()  # or interpolate(method="time")



# # Convert yields from % to decimals (optional but usually nicer)
ccc = ccc / 100.0

t10 = t10 / 100.0

spread_CCC = ccc - t10
spread_CCC.name = "CCC_10Y_Spread"
```

2 Market Variables

[ ]:
```
# VIX (daily → monthly)
vix_daily = yf.download("^VIX", start=START_DATE, end=END_DATE, session=session)
vix_daily = vix_daily["Close"]
vix_daily.index = make_tz_naive(vix_daily.index)
vix_daily.name = "VIX"
vix_daily = vix_daily.reindex(pd.date_range(t10.index.min(), t10.index.max(),␣
 ↪freq="D"))
vix_daily = vix_daily.interpolate(method='time')
vix = vix_daily

# S&P500 price and monthly returns (proper monthly returns)
sp500_daily = yf.download("^GSPC", start=START_DATE, end=END_DATE,␣
 ↪session=session)
sp500_daily = sp500_daily["Close"]
sp500_daily.index = make_tz_naive(sp500_daily.index)
sp500_daily.name = "SP500_Close"
sp500_daily = sp500_daily.reindex(pd.date_range(t10.index.min(), t10.index.
 ↪max(), freq="D"))
sp500_daily = sp500_daily.interpolate(method='time')

sp500_monthly = sp500_daily
```

```
sp500_ret = sp500_monthly.pct_change()
sp500_ret.name = "SP500_Return"
```

```
1 Failed download:
['^VIX']: AttributeError("'str' object has no attribute 'name'")
[**********************100%**********************]  1 of 1 completed

1 Failed download:
['^GSPC']: AttributeError("'str' object has no attribute 'name'")
/var/folders/wt/7k1qgpyn1n1c3z8mgd_rmz3c0000gn/T/ipykernel_40158/1630830224.py:2
6: FutureWarning: The default fill_method='pad' in DataFrame.pct_change is
deprecated and will be removed in a future version. Either fill in any non-
leading NA values prior to calling pct_change or specify 'fill_method=None' to
not fill NA values.
  sp500_ret = sp500_monthly.pct_change()
```

3 Macro/ Yield_curve

[14]:
```
# 2Y Treasury
t2 = fred.get_series("GS2", observation_start=START_DATE,␣
 ↪observation_end=END_DATE)
t2.index = pd.to_datetime(t2.index)
t2 = t2
t2.name = "T2Y"
t2 = t2.reindex(pd.date_range(t10.index.min(), t10.index.max(), freq="D"))
t2 = t2.interpolate(method='time')

t2 = t2 / 100.0  # percent to decimal

yield_curve_slope = t10 - t2
yield_curve_slope.name = "YieldCurveSlope"

# Unemployment rate (monthly already, but resample for safety)
unemp = fred.get_series("UNRATE", observation_start=START_DATE,␣
 ↪observation_end=END_DATE, frequency="m")
unemp.index = pd.to_datetime(unemp.index)
unemp = unemp
unemp.name = "Unemployment"
unemp = unemp.reindex(pd.date_range(t10.index.min(), t10.index.max(), freq="D"))
unemp = unemp.interpolate(method='time')

# CPI and inflation (monthly)
cpi = fred.get_series("CPIAUCSL", observation_start=START_DATE,␣
 ↪observation_end=END_DATE, frequency="m")
cpi.index = pd.to_datetime(cpi.index)
```

```
inflation = cpi.pct_change()  # month-over-month inflation
inflation.name = "Inflation"
inflation = inflation.reindex(pd.date_range(t10.index.min(), t10.index.max(),␣
 ↪freq="D"))
inflation = inflation.interpolate(method='time')


# GDP (quarterly → monthly, forward-filled, then growth)
gdp = fred.get_series("GDP", observation_start=START_DATE,␣
 ↪observation_end=END_DATE, frequency="q")
gdp.index = pd.to_datetime(gdp.index)
# Quarterly to monthly with forward-fill
gdp_m = gdp
gdp_growth = gdp_m.pct_change()
gdp_growth.name = "GDP_Growth"
gdp_growth = gdp_growth.reindex(pd.date_range(t10.index.min(), t10.index.max(),␣
 ↪freq="D"))
gdp_growth = gdp_growth.interpolate(method='time')

indpro = fred.get_series("INDPRO", observation_start=START_DATE,␣
 ↪observation_end=END_DATE, frequency="m")
indpro.index = pd.to_datetime(indpro.index)
indpro_growth = indpro.pct_change()
indpro_growth.name = "IndPro_Growth"
indpro_growth = indpro_growth.reindex(pd.date_range(t10.index.min(), t10.index.
 ↪max(), freq="D"))
indpro_growth = indpro_growth.interpolate(method='time')
```

```
/var/folders/wt/7k1qgpyn1n1c3z8mgd_rmz3c0000gn/T/ipykernel_40158/3585602968.py:2
5: FutureWarning: The default fill_method='pad' in Series.pct_change is
deprecated and will be removed in a future version. Either fill in any non-
leading NA values prior to calling pct_change or specify 'fill_method=None' to
not fill NA values.
  inflation = cpi.pct_change()  # month-over-month inflation
```

4 Firm Level

[15]:
```
cp = fred.get_series("CP", observation_start=START_DATE,␣
 ↪observation_end=END_DATE, frequency="q")
cp.index = pd.to_datetime(cp.index)
cp_growth = cp.pct_change()
cp_growth.name = "CP_Growth"
cp_growth = cp_growth.reindex(pd.date_range(t10.index.min(), t10.index.max(),␣
 ↪freq="D"))
cp_growth = cp_growth.interpolate(method='time')
```

```
# Corporate leverage proxy (total credit market debt / something)
corp_lev_raw = fred.get_series("TCMDO", observation_start=START_DATE,␣
 ↪observation_end=END_DATE, frequency="q")
corp_lev_raw.index = pd.to_datetime(corp_lev_raw.index)
corp_lev_m = corp_lev_raw
corp_leverage = corp_lev_m.pct_change()
corp_leverage.name = "Corporate_Leverage"
corp_leverage = corp_leverage.reindex(pd.date_range(t10.index.min(), t10.index.
 ↪max(), freq="D"))
corp_leverage = corp_leverage.interpolate(method='time')
```

[ ]:
```
df = pd.concat(
    [
        spread_CCC,
        vix,
        sp500_ret,
        yield_curve_slope,
        unemp,
        inflation,
        gdp_growth,
        corp_leverage,
        indpro_growth,
        cp_growth,
    ],
    axis=1,
)

# print(df.head())


# df.set_index('Date')
# df = df.interpolate(method='linear')

# # Delete the first row

# # Combine all into a single DataFrame
# df.to_csv("data.csv")

# print(df.shape)
# print(df.head())
```